



Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

Gestione dell'I/O in Java Complementi

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



INDICE

- Questi complementi affrontano le seguenti tematiche:
 1. **Stream di adattamento** fra stream binari e stream di testo
 - caso particolare: **System.in**
 2. **La libreria & factory Files**
 - metodi per leggere / scrivere tutte le righe di piccoli fili di testo
 - metodi factory per la costruzione veloce degli stream più usati
 3. **Text-based persistence**
 - tematica generale, cenni ai formati più diffusi
 - casi particolari: i nostri formati d'esame e il formato CSV
 4. **Letture di file contenuti in archivi JAR**
 - tecniche per la lettura di file inglobati nel JAR dell'applicazione

1. Stream di byte vs testo: adapter fra i due mondi



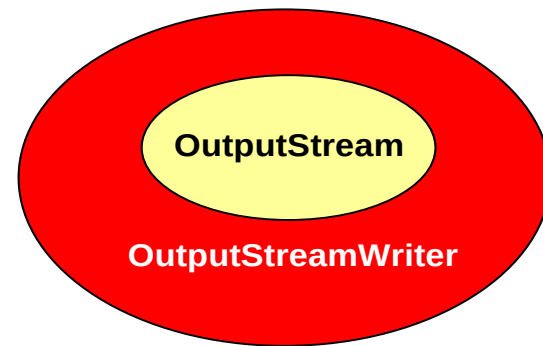
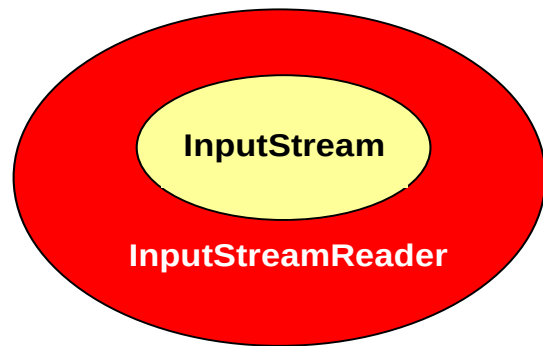
STREAM BINARI O DI TESTO ?

- Finora abbiamo considerato la lettura / scrittura *binaria* come nettamente distinta dalla lettura / scrittura *di testo*
- Tuttavia, la suddivisione non è sempre così netta:
 - la sorgente può emettere *tipi di dati diversi in momenti diversi* (es: prima testo, poi immagini, poi altro testo)
 - stream di testo «non Java» possono apparire binari
- Inoltre, se i byte in transito in quel momento sono testo, ha senso poter fare letture/scritture *per righe*

Può essere necessario
*trattare uno stream binario
come stream di testo*

UN ADAPTER FRA I DUE MONDI

- A tal fine, due *stream di adattamento* incapsulano *uno stream di byte in uno stream di testo*

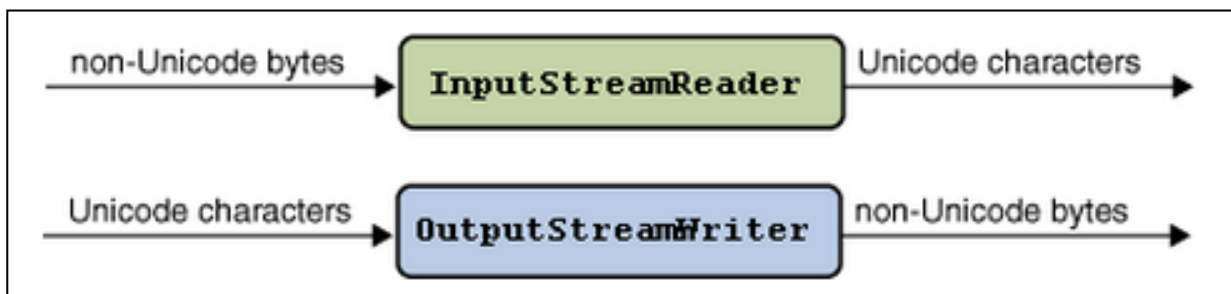


- ***InputStreamReader*** ingloba un ***InputStream*** e lo fa apparire all'esterno come un ***Reader***
- ***OutputStreamWriter*** ingloba un ***OutputStream*** e lo fa apparire fuori come un ***Writer***

UN ADAPTER FRA I DUE MONDI

Più in generale, i due adapter convertono

- IN LETTURA, sequenze di *byte non-Unicode* in sequenze corrette di **caratteri Unicode**
- IN SCRITTURA, sequenze corrette di **caratteri Unicode** in sequenze di *byte non-Unicode*





Il caso tipico: TASTIERA & VIDEO

- Video e tastiera sono legati ai due stream standard (statici) **System.in** e **System.out**
- Essi sono formalmente degli stream di byte, perché risalgono a Java 1.0: però, *di solito si usano a caratteri*
 - CONSEGUEENZA: non è detto che i caratteri Unicode vengano interpretati correttamente
 - MOTIVO: negli stream di byte non esiste il concetto fondamentale di *character encoding* (poi *Charset*)
 - RISULTATO: rischio di *comportamenti "bizzarri"* in presenza di lettere accentate e altri caratteri "strani"
- Per questi motivi, il loro uso da console (prompt dei comandi) dovrebbe *sempre* essere filtrato dall'opportuno *adapter*.



UN ESEMPIO DI "BIZZARRIA"

- Si supponga di voler leggere da tastiera una stringa *contenente lettere accentate*
 - ad esempio, il giorno della settimana "**venerdì**"
- e di volerlo poi *ristampare a video*.

COME LEGGERE DA TASTIERA?

- **System.in** è un **InputStream**, non ha metodi adatti
- Il vecchio adapter **DataInputStream** offre un metodo **readLine** che però è *giustamente deprecato* perché *"does not properly convert bytes to characters"*
- Il modo corretto è interporre un **InputStreamReader** correttamente configurato (o usare la classe **Console**)



ESEMPIO con `System.in`

- Con il vecchio adapter deprecato:

```
System.out.print("Inserire giorno: ");
```

```
DataStream in = new DataInputStream(System.in);
```

```
String giorno = in.readLine();
```

```
System.out.println(giorno);
```

- Output:

```
C:>java TestAccenti  
Inserire giorno: venerdì  
venerd?
```

BELLO, VERO?

Il metodo `readLine` di
`DataInputStream`
**sbaglia la conversione in
Unicode
→ DEPRECATO**

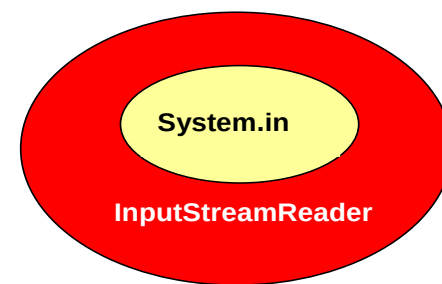
INCAPSULARE L'INPUT

- La cura è adattare `System.in` a *stream di testo*:

```
InputStreamReader myIn =  
    new InputStreamReader(System.in, ...);
```

- A sua volta, costui sarà incapsulato in un `BufferedReader` per avere `readLine`:

```
InputStreamReader tastiera =  
    new BufferedReader(  
        new InputStreamReader(System.in, ...) );
```





L'ESEMPIO RIVISTO

- Esempio ristrutturato:

```
BufferedReader in = new BufferedReader(  
    new InputStreamReader(System.in, "CP850"));
```

Il giusto encoding

```
String giorno = in.readLine();  
System.out.println(giorno);
```

Il metodo `readLine` di `BufferedReader` opera correttamente in Unicode

- Output:

```
C:>java TestAccenti  
Inserire giorno: venerdì  
venerdì
```

Funziona!

CP850 è la codifica standard delle finestre «prompt dei comandi» in Windows versione italiana.

```
>chcp  
Tabella codici attiva: 850
```



L'ESEMPIO RIVISTO

- Attenzione a quali finestre di output usate
 - **CP 850** è l'encoding dei terminali «prompt comandi»,
non di tutte le finestre Windows!
 - come è facile scoprire, l'encoding standard delle
finestre Windows è **CP1252**:

Cp1252

```
System.out.println(  
    new InputStreamReader(System.in).getEncoding());
```

- Perciò, nel caso Windows:
 - se l'applicazione deve funzionare *da terminale*,
occorre impostare **CP 850**
 - se invece deve funzionare *da finestra grafica*,
va bene l'encoding di default **CP 1252**



L'ESEMPIO con Console

- Esempio ristrutturato:

```
Console console = System.console();  
String giorno = console.readLine();  
System.out.println(giorno);
```

Il giusto encoding

- Output:

Il metodo `readLine` di
`Console` opera
correttamente in Unicode

```
C:>java TestAccenti  
Inserire giorno: venerdì  
venerdì
```

Funziona!

NB: ricordare che `Console` può essere *null* se non c'è un terminale «adatto», ossia se la JVM è stata lanciata da `javaw` anziché `java`



INCAPSULARE L'OUTPUT

- Analogamente, si può adattare `System.out`:

```
OutputStreamWriter video =  
    new OutputStreamWriter(System.out, ...);
```

da incapsulare poi, magari, in un `PrintWriter`.

- La cosa è meno grave del caso input, perché in Java 1.0 `System.out` era un `PrintStream`, meno problematico.

```
PrintWriter video =  
    new PrintWriter(new OutputStreamWriter(  
        System.out, "CP850"), true);
```

Di nuovo, `Console` (se disponibile) risolve molti problemi.



CHARSET DISPONIBILI

- Un **Charset** definisce un *mapping byte/caratteri*
- Per sapere quali sono quelli disponibili:

```
System.out.print(Charset.availableCharsets())
```

```
Big5, Big5-HKSCS, CESU-8, EUC-JP, EUC-KR, GB18030, GB2312, GBK, IBM-Thai, IBM00858, IBM01140,
IBM01141, IBM01142, IBM01143, IBM01144, IBM01145, IBM01146, IBM01147, IBM01148, IBM01149, IBM037,
IBM1026, IBM1047, IBM273, IBM277, IBM278, IBM280, IBM284, IBM285, IBM290, IBM297, IBM420, IBM424,
IBM437, IBM500, IBM775, IBM850, IBM852, IBM855, IBM857, IBM860, IBM861, IBM862, IBM863, IBM864,
IBM865, IBM866, IBM868, IBM869, IBM870, IBM871, IBM918, ISO-2022-CN, ISO-2022-JP, ISO-2022-JP-2,
ISO-2022-KR, ISO-8859-1, ISO-8859-13, ISO-8859-15, ISO-8859-2, ISO-8859-3, ISO-8859-4, ISO-8859-5,
ISO-8859-6, ISO-8859-7, ISO-8859-8, ISO-8859-9, JIS_X0201, JIS_X0212-1990, KOI8-R, KOI8-U,
Shift_JIS, TIS-620, US-ASCII, UTF-16, UTF-16BE, UTF-16LE, UTF-32, UTF-32BE, UTF-32LE, UTF-8,
windows-1250, windows-1251, windows-1252, windows-1253, windows-1254, windows-1255, windows-1256,
windows-1257, windows-1258, windows-31j, x-Big5-HKSCS-2001, x-Big5-Solaris, x-euc-jp-linux, x-EUC-
TW, x-eucJP-Open, x-IBM1006, x-IBM1025, x-IBM1046, x-IBM1097, x-IBM1098, x-IBM1112, x-IBM1122, x-
IBM1123, x-IBM1124, x-IBM1364, x-IBM1381, x-IBM1383, x-IBM300, x-IBM33722, x-IBM737, x-IBM833, x-
IBM834, x-IBM856, x-IBM874, x-IBM875, x-IBM921, x-IBM922, x-IBM930, x-IBM933, x-IBM935, x-IBM937,
x-IBM939, x-IBM942, x-IBM942C, x-IBM943, x-IBM943C, x-IBM948, x-IBM949, x-IBM949C, x-IBM950, x-
IBM964, x-IBM970, x-ISCII91, x-ISO-2022-CN-CNS, x-ISO-2022-CN-GB, x-iso-8859-11, x-JIS0208, x-
JISAutoDetect, x-Johab, x-MacArabic, x-MacCentralEurope, x-MacCroatian, x-MacCyrillic, x-
MacDingbat, x-MacGreek, x-MacHebrew, x-MacIceland, x-MacRoman, x-MacRomania, x-MacSymbol, x-
MacThai, x-MacTurkish, x-MacUkraine, x-MS932_0213, x-MS950-HKSCS, x-MS950-HKSCS-XP, x-mswin-936, x-
PCK, x-SJIS_0213, x-UTF-16LE-BOM, x-UTF-32BE-BOM, x-UTF-32LE-BOM, x-windows-50220, x-windows-50221,
x-windows-874, x-windows-949, x-windows-950, x-windows-iso2022jp
```

2. La libreria & factory **Files**



LA LIBRERIA & FACTORY `Files`

- `java.nio.file.Files` è una libreria & factory
 1. metodi per operare su file e directory (già discussi)
 - copiare o spostare file o directory
 - creare / eliminare file, directory, link simbolici
 - creare file o directory temporanee
 - verificare l'esistenza di file o directory
 - recuperare attributi di file o directory
 2. metodi per leggere e scrivere piccoli file
 - leggere / scrivere tutti i byte di un piccolo file binario (già discussi)
 - leggere / scrivere tutte le *righe* di un piccolo file di testo
 3. metodi factory per *creare stream di I/O*
 - per velocizzare la creazione degli stream più usati (convenience methods non essenziali, solo di comodità)



LA LIBRERIA & FACTORY Files

- Per *leggere* piccoli file in blocco ci sono i metodi
 - **Files.readAllBytes** per leggere tutti i byte di un file binario
 - **Files.readAllLines** per leggere tutte le righe di un file di testo
 - riceve il percorso e il Charset
 - restituisce una lista di stringhe (righe)

readAllLines

```
public static List<String> readAllLines(Path path,  
                                       Charset cs)  
    throws IOException
```

Read all lines from a file. This method ensures that the file is closed when all bytes have been read or an I/O error, or other runtime exception, is thrown. Bytes from the file are decoded into characters using the specified charset.

This method recognizes the following as line terminators:

- `\u000D` followed by `\u000A`, CARRIAGE RETURN followed by LINE FEED
- `\u000A`, LINE FEED
- `\u000D`, CARRIAGE RETURN

Additional Unicode line terminators may be recognized in future releases.

Note that this method is intended for simple cases where it is convenient to read all lines in a single operation. It is not intended for reading in large files.



LA LIBRERIA & FACTORY Files

- Per *scrivere* piccoli file in blocco ci sono i metodi
 - **Files.write** per scrivere tutti i byte di un file binario
 - **Files.write** per scrivere tutte le righe di un file di testo
 - riceve il percorso, l'insieme (iterabile) di righe, il Charset ed eventualmente una o più opzioni di apertura
 - restituisce il percorso stesso

write

```
public static Path write(Path path,  
                        Iterable<? extends CharSequence> lines,  
                        Charset cs,  
                        OpenOption... options)  
    throws IOException
```

Write lines of text to a file. Each line is a char sequence and is written to the file in sequence with each line terminated by the platform's line separator, as defined by the system property `line.separator`. Characters are encoded into bytes using the specified charset.

The options parameter specifies how the file is created or opened. If no options are present then this method works as if the `CREATE`, `TRUNCATE_EXISTING`, and `WRITE` options are present. In other words, it opens the file for writing, creating the file if it doesn't exist, or initially truncating an existing regular-file to a size of 0. The method ensures that the file is closed when all lines have been written (or an I/O error or other runtime exception is thrown). If an I/O error occurs then it may do so after the file has been created or truncated, or after some bytes have been written to the file.



LAVORARE SU PICCOLI FILE (1)

- Lettura di un *piccolo* file di testo

```
public static void main(String args[]){  
    try {  
        List<String> rows =  
            Files.readAllLines(Paths.get("dante.txt"));  
        System.out.println("Lette " + rows.size() + " righe");  
    }  
    catch(IOException e){  
        System.out.println("Problema di lettura");  
        System.exit(1);  
    }  
}
```

Lette 3 righe

se il file esiste ed è stato
aperto regolarmente

Problema di lettura

se il file non esiste o non
può essere aperto

MAI usare questo approccio per file "non piccoli" !

LAVORARE SU PICCOLI FILE (2)

- Scrittura di un *piccolo* file di testo

```
public static void main(String args[]){  
    try {  
        String[] righe = {"Nel mezzo", "del cammin di nostra vita",  
                           "mi ritrovai", "per una selva oscura",  
                           "che la diritta via", "era smarrita." };  
        Files.write(Paths.get("dante2.txt"), Arrays.asList(righe));  
        System.out.println("Scritte " + righe.length + " righe");  
    }  
    catch(IOException e){  
        System.out.println("Problema di scrittura");  
        System.exit(1);  
    }  
}
```

Scritte 6 righe

se il file esiste ed è stato
aperto regolarmente

Problema di scrittura

se il file non esiste o non
può essere aperto

MAI usare questo approccio per file "non piccoli" !



LA FABBRICA DEGLI STREAM

- Come già discusso per gli stream binari, anche per gli stream di testo la factory **Files** offre *factory methods* di utilità per velocizzare la costruzione
 - tali metodi producono già un **BufferedReader/Writer** evitando di dover specificare gli strati della «cipolla» ☺

```
newBufferedReader(Path p)
```

```
newBufferedReader(Path p, Charset cs)
```

```
newBufferedWriter(Path p, Charset cs, OpenOption... opts)
```

```
newBufferedWriter(Path p, OpenOption... opts)
```

dove **Charset** rappresenta lo specifico *set di caratteri* da usare e la notazione *varargs* (**Tipo...**) indica zero o più argomenti di quel certo **Tipo**

- valgono considerazioni ed esempi analoghi a quelli già mostrati per gli stream binari

3. Text-based persistence

PERSISTENZA TEXT-BASED

- Spesso, anziché in formato binario, si preferisce **salvare i dati in formato testo**
 - PRO: leggibilità, fruibilità, indipendenza dalla piattaforma
 - CONTRO: talora qualche difficoltà in più, soprattutto in lettura
- Si parla in questi casi di «**serializzazione in formato testo**»

From Wikipedia: *serialization is the process of translating data structures or object state into a format that can be stored (for example, in a file or memory buffer, or transmitted across a network connection link) and reconstructed later in the same or another computer environment*

- Formati tipici vs. formati «custom»
 - XML (famiglia di formati), CSV (Excel, etc.), JSON
 - formati «custom»: qualunque protocollo progettato ad hoc

PERSISTENZA TEXT-BASED: PUNTI CHIAVE

- Necessità di stabilire un **protocollo preciso**
 - tutti i tipi di dati devono essere opportunamente scritti e letti
 - tutto è stringa, progettato per essere facilmente ri-leggibile
 - in scrittura, ogni oggetto dev'essere compiutamente rappresentato
 - in lettura, occorre ri-produrre *oggetti* usabili da programma
- Necessità di porre **particolare attenzione** a:
 - **formati dei numeri**: separatori di migliaia e decimali?
 - **formati delle date/ore**: tipo di formattazione, separatori, ecc.
 - **separatori dei vari campi**: quali (gruppi di) caratteri usare?

PERSISTENZA TEXT-BASED: IL CASO XML

- **XML** è un linguaggio di markup, basato su una struttura di *tag annidati* della forma **<TAG> ... </TAG>**
 - con opportuna scelta dei tag, si può descrivere qualunque cosa
 - la struttura annidata dà luogo a un albero
 - ampia disponibilità di strumenti e riconoscitori (XML parser)

```
<Text text="Welcome"
      GridPane.columnIndex="0" GridPane.rowIndex="0"
      GridPane.columnSpan="2"/>
<Label text="User Name:"
      GridPane.columnIndex="0" GridPane.rowIndex="1"/>
<TextField
      GridPane.columnIndex="1" GridPane.rowIndex="1"/>
<Label text="Password:"
      GridPane.columnIndex="0" GridPane.rowIndex="2"/>
<PasswordField fx:id="passwordField"
      GridPane.columnIndex="1" GridPane.rowIndex="2"/>
```

PERSISTENZA TEXT-BASED: IL CASO JSON

- **JSON** (JavaScript Object Notation) è un linguaggio basato sulla notazione a oggetti di JavaScript
 - con opportuna scelta della struttura degli oggetti (in termini di proprietà), si può descrivere qualunque cosa
 - la struttura è di base semplice, ma può essere nidificata
 - ampia disponibilità di strumenti e riconoscitori

```
{  
  "nome": "Mario",  
  "cognome": "Rossi",  
  "dataNascita": {  
    "giorno": 15,  
    "mese": 4,  
    "anno": 2000  
  },  
  "nazione": "Italia"  
}
```



PERSISTENZA TEXT-BASED: FORMATI «CUSTOM»

- I formati «custom» sono formati non standard, inventati e usati entro applicazioni specifiche → **i nostri esami** 😊

- ESEMPIO: Voti alle elezioni

SEGGI 20

Lista A 9.100.000, Lista E 1.320.000, Lista C 4.880.000, Lista D 4.100.000, Lista B 1.500.000

- ESEMPIO: Linee di autobus con tempi di percorrenza e fermate

Linea 32

0, 40, Porta San Mamolo

3, 42, Aldini

5, 44, Porta Saragozza - Villa Cassarini

17, 16, Stazione Centrale

- ESEMPIO: Tracking di un volo aereo

UTC;Position;Altitude;Speed;Direction

2019-05-10T10:54:39Z;45.661972,8.726303;1975;183;356

2019-05-10T10:54:49Z;45.67049,8.725822;2450;182;358

...

PERSISTENZA TEXT-BASED: IL CASO «CSV»

- CSV = Comma Separated Values
 - in teoria, la cosa più standard dell'universo: la virgola è la virgola!
 - in realtà, spesso si intende non la «virgola» in sé (concetto assoluto) ma il *separatore di elenco* (concetto relativo, locale)
 - a volte si intendono anche «tabulazioni» o spazi o altro
- Per l'Italia, il separatore di elenco è «;», non «,»
 - questo può portare a risultati inattesi: elenchi esportati con il Locale inglese possono dare errore quando riletti con il Locale italiano
- Ovviamente, il separatore scelto *non* può far parte dei dati
 - in caso sia presente, andrà sostituito da qualcos'altro o «escaped» con qualche carattere speciale → *formati specifici*

ESEMPIO: EXCEL «CSV»

- Caso tipico: Excel

Esporta

 Crea documento PDF/XPS

 **Cambia tipo di file**

Cambia tipo di file

Tipi di file delle cartelle di lavoro

 Cartella di lavoro (*.xlsx) Usa il formato foglio di calcolo di Excel	 Cartella di lavoro di Excel 97-2003 (*.xls) Usa il formato foglio di calcolo Excel 97-2003
 Formato ODS (*.ods) Usa il formato ODS	 Modello (*.xltx) Punto di partenza per i nuovi fogli di calcolo
 Cartella di lavoro con attivazione macro... Foglio di calcolo con attivazione macro	 C...

Altri tipi di file

 Testo (con valori delimitati da tabulazio... Formato testo separato da tabulazioni	 CSV (delimitato dal separatore di elenco... Formato testo con valori separati da virgole
 Testo formattato (delimitato da spazio)... Formato testo con valori separati da spazi	 Salva come altro tipo di file

Sono CSV anche i tab-separated

Sono CSV anche i o space-separated

Virgole..?!?

ESEMPIO: EXCEL «CSV»

- Caso tipico: Excel

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Matricola	Cognome	Nome	codice fiscale	data di nascita	luogo di nascita	provincia	stato	indirizzo	città	provincia	stato	cap
2	12345678	Rossi	Mario	RSSMRA76H12A944I	12/06/1976	Bologna	BO	Italia	via Indipendenza 22	Bologna	BO	Italia	40121
3	12345699	Rossi	Mario	RSSMRA90A01F205Z	01/01/1990	Milano	MI	Italia	piazza del Duomo 1	Milano	MI	Italia	20121

Nome file:

Salva come: CSV delimitato dal separatore di elenco (*.csv)

- il separatore di elenco dipende dalle impostazioni di località
→ per l'Italia, è il «;»

```
Matricola;Cognome;Nome;codice fiscale;data di nascita;luogo di nascita;provincia;stato;indirizzo;città;provincia;stato;cap
12345678;Rossi;Mario;RSSMRA76H12A944I;12/06/1976;Bologna;BO;Italia;via Indipendenza 22;Bologna;BO;Italia;40121
12345699;Rossi;Mario;RSSMRA90A01F205Z;01/01/1990;Milano;MI;Italia;piazza del Duomo 1;Milano;MI;Italia;20121
```

E se un dato conteneva già il «;» ?

ESEMPIO: EXCEL «CSV»

- Caso tipico: Excel con un dato contenente il separatore

	A	B	C	D	E	F	G	H	I	J
1	Matricola	Cognome	Nome	codice fiscale	data di nascita	luogo di nascita	provincia	stato	indirizzo	città
2	12345678	Rossi	Mario	RSSMRA76H12A944I	12/06/1976	Bologna	BO	Italia	via Indipendenza 22; Torre Asinelli 1	Bologna
3	12345699	Rossi	Mario	RSSMRA90A01F205Z	01/01/1990	Milano	MI	Italia	piazza del Duomo 1	Milano

- Cosa succede a quel «;» ?

- nel caso di Excel, quando il file viene salvato in CSV, quel campo dati viene virgolettato

```
e;Nome;codice fiscale;data di nascita;luogo di nascita;provincia;stato;indirizzo;città;provincia;s
ario;RSSMRA76H12A944I;12/06/1976;Bologna;BO;Italia;"via Indipendenza 22; Torre Asinelli 1";Bologna
ario;RSSMRA90A01F205Z;01/01/1990;Milano;MI;Italia;piazza del Duomo 1;Milano;MI;Italia;20121
```

- MA è una scelta di questo specifico formato!
- altri avrebbero potuto anteporre una «\», o inventarsi qualcos'altro
- Importante verificare *sempre* la specifica *esatta* del formato!

4. Tecniche per la lettura di file inglobati nel JAR dell'applicazione



ARCHIVI JAR: DATI vs. RISORSE

- Gli archivi JAR si usano per distribuire *librerie* o *applicazioni*
- Se tali software devono accedere a file, occorre distinguere:
 - JAR che devono accedere a file **esterni** al JAR stesso
 - JAR che devono accedere a file **interni** al JAR stesso
- Tipicamente:
 - un'applicazione che elabori **file di utente** non li avrà, ovviamente, inclusi nel JAR stesso
 - un'applicazione che invece carichi **dati, icone o suoni per la propria configurazione** li avrà al proprio interno
 - si parla più propriamente di **risorse**

ARCHIVI JAR: DATI vs. RISORSE

- Le classi **FileReader** e **FileInputStream** possono accedere **solo a file esterni** al JAR stesso
 - motivo: sono pensati per accedere al *file system*
 - *non sono in grado* di accedere a *risorse interne* al JAR
- Per leggere file di **risorse** (testo, immagini, suoni) **interne** al JAR, occorre utilizzare il **metodo getResourceAsStream** della classe in cui si fa la lettura
 - **getResourceAsStream** restituisce un **InputStream**, che nel caso di file di testo potrà essere utilmente incapsulato in un **InputStreamReader** per apparire come **Reader**
 - più esattamente, **getResourceAsStream** è un metodo della (meta) classe Class, i cui oggetti rappresentano le classi presenti nel sistema a run-time: si usa perciò secondo uno *schema particolare*

getResourceAsStream

- L'oggetto di classe **Class** su cui invocare il metodo **getResourceAsStream** si ottiene:
 - invocando il metodo **getClass ()** sull'oggetto corrente,
se il metodo in cui si sta lavorando *non è statico*
ESEMPIO:
this.getClass ().getResourceAsStream (...)
 - con la notazione **Nomeclasse.class**,
se il metodo in cui si sta lavorando *è statico*
ESEMPIO:
MyClass.class.getResourceAsStream (...)
 - NB: il percorso nel metodo **getResourceAsStream** si riferisce alla *root dello zip file*

ESEMPIO 1:

LETTURA DI DATI BINARI

- Si supponga che il file binario **resource.bin** contenga valori interi, precedentemente salvati
 - non è dato sapere a priori quanti siano, quindi occorrerà per forza effettuare un ciclo di lettura «alla cieca» fino a EOFException
- Versione classica, con file esterno all'eventuale JAR:

```
try (DataInputStream is = new DataInputStream(new FileInputStream(filename))) {  
    while(true) System.out.println(is.readInt());  
    // eventually, it will be interrupted by EOF  
}  
catch(FileNotFoundException e1){  
    System.out.println("File not found: " + filename);  
    System.exit(2);  
}  
catch(EOFException e2){  
    // normal termination, actually  
    System.exit(0);  
}  
catch (IOException e3){  
    System.out.println("Unexpected input error");  
    System.exit(3);  
}
```

ESEMPIO 1:

LETTURA DI DATI BINARI

- Se il file binario `resource.bin` deve essere incluso in un JAR come risorsa interna, la lettura va riformulata
 - anziché costruire un `FileInputStream`, occorre recuperare un input stream come risorsa tramite `getResourceAsStream`

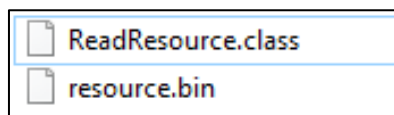
```
public class ReadResource {  
  
    public static void main(String args[]){  
        String filename = "resource.bin";  
  
        try (DataInputStream is = new DataInputStream(ReadResource.class.getResourceAsStream(filename))) {  
            while(true) System.out.println(is.readInt());  
            // eventually, it will be interrupted by EOF  
        }  
        catch(FileNotFoundException e1){  
            System.out.println("File not found: " + filename);  
            System.exit(2);  
        }  
        catch(EOFException e2){  
            // normal termination, actually  
            System.exit(0);  
        }  
        catch (IOException e3){  
            System.out.println("Unexpected input error");  
            System.exit(3);  
        }  
    }  
}
```

Usa la notazione
NomeClasse.class
perché questo codice si trova
in un metodo statico (il main)

ESEMPIO 1:

LETTURA DI DATI BINARI

- Dopo aver testato il corretto funzionamento, si può predisporre il JAR autocontenuto



```
jar cef ReadResource ReadData.jar ReadResource.class resource.bin
```

Nome classe
con il main

Nome JAR
da produrre

File da
includere

- Test del JAR autocontenuto appena creato
 - NB: da fare senza nessun altro file nella cartella!



```
C:\Temp\prova>java -jar ReadData.jar
23
-5
61
18
```

ESEMPIO 2: LETTURA DI UN FILE DI TESTO

- Main originale (statico) che accedeva a un file su disco:

```
try(FileReader rdr =  
    new FileReader("LineaMI-BO.txt")) {  
    ...  
}
```

- Main modificato per accedere a un file interno al JAR:

```
try(InputStreamReader rdr =  
    new InputStreamReader(  
        MyMain.class.getResourceAsStream(  
            "/LineaMI-BO.txt"))) {  
    ...  
}
```

Usa la notazione
NomeClasse.class
perché questo codice si trova
in un metodo statico (il main)

Il percorso **/** si riferisce
alla *root dello zip file*

ESEMPIO 2 – VERSIONE ORIGINALE

(file da leggere esterno al JAR)

```
>java -jar ferrovia.jar  
[Milano Centrale - km 0, Milano Lambrate - km 4, Milano Rogoredo - km 10, Piacen  
za - km 72, Fidenza - km 107, Parma - km 129, Reggio Emilia - km 157, Modena - k  
m 182, Bologna Centrale - km 219]  
Milano Centrale - km 0  
Milano Lambrate - km 4  
Milano Rogoredo - km 10  
Piacenza - km 72  
Fidenza - km 107  
Parma - km 129  
Reggio Emilia - km 157  
Modena - km 182  
Bologna Centrale - km 219  
Milano Centrale - km 0  
Milano Lambrate - km 4  
Milano Rogoredo - km 10  
Piacenza - km 72  
Fidenza - km 107  
Parma - km 129  
Reggio Emilia - km 157  
Modena - km 182  
Bologna Centrale - km 219
```

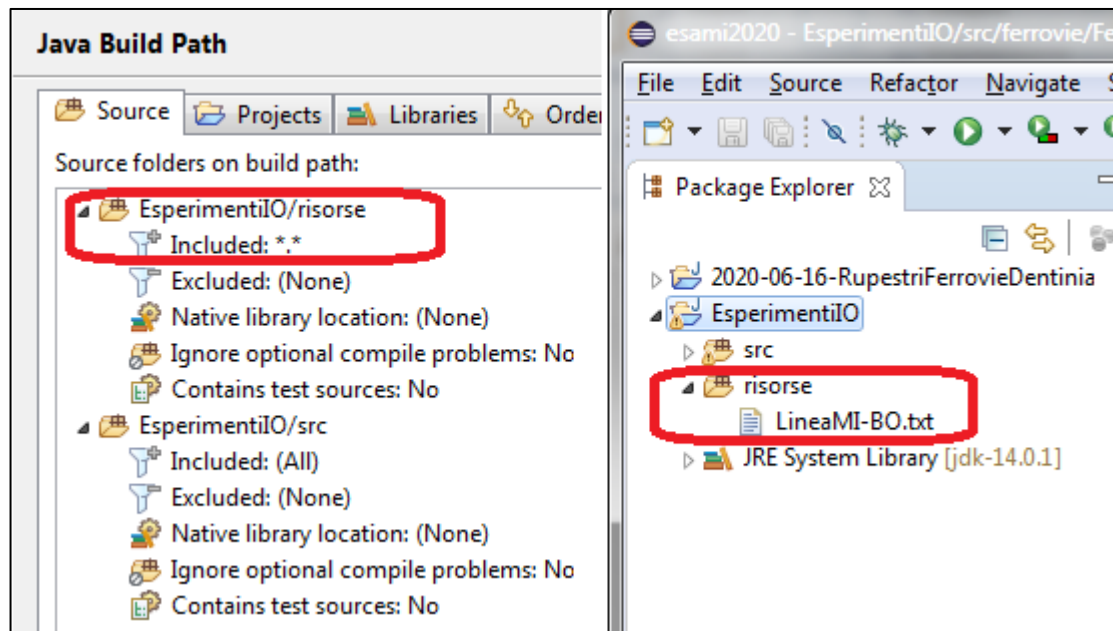
Situazione classica: il file da leggere è esterno al JAR
→ si usa normalmente FileReader

Infatti, il file di testo *non* è incluso nel JAR

```
>jar -tf ferrovia.jar jar  
META-INF/MANIFEST.MF  
ferrovie/  
ferrovie/Ferrovia.class  
ferrovie/BadFileFormatException.class  
ferrovie/FerroviaConScanner.class  
ferrovie/Stazione.class  
ferrovie/FerroviaBinaryPersister.class  
persone/  
persone/BadFileFormatException.class  
persone/Persona.class  
persone/LeggiPersone.class  
persone/LeggiPersoneConScanner.class  
persone/LeggiPersoneConSplit.class  
voli/  
voli/Flight.class  
voli/FlightPos.class  
voli/BadFileFormatException.class  
voli/LeggiVolo.class
```

CREAZIONE DEL JAR CON RISORSE IN ECLIPSE

- Da Eclipse, far includere i file-risorse richiede un po' di cura
 1. creare una cartella apposita, di tipo *source folder*, nel progetto (es. risorse) e *metterci dentro i file necessari*
 2. aggiungerla al *build path*
 3. specificare come *include pattern* di quella cartella il tipo di file da includere (es. tutti: *.*)
 4. Esportare normalmente il *Runnable jar*



ESEMPIO 2 – VERSIONE MODIFICATA

(file da leggere *interno* al JAR)

```
>jar -tf ferrovia2.jar
META-INF/MANIFEST.MF
org/
org/eclipse/
org/eclipse/jdt/
org/eclipse/jdt/internal/
org/eclipse/jdt/internal/jarinjarloader/
org/eclipse/jdt/internal/jarinjarloader/JIJConstants.class
org/eclipse/jdt/internal/jarinjarloader/JarRsrcLoader$ManifestInfo.class
org/eclipse/jdt/internal/jarinjarloader/JarRsrcLoader.class
org/eclipse/jdt/internal/jarinjarloader/RsrcURLConnection.class
org/eclipse/jdt/internal/jarinjarloader/RsrcURLStreamHandler.class
org/eclipse/jdt/internal/jarinjarloader/RsrcURLStreamHandlerFactory.class
ferrovie/
ferrovie/FerroviaMainResource.class
ferrovie/FerroviaMain.class
ferrovie/BadFileFormatException.class
ferrovie/FerroviaConScanner.class
ferrovie/Stazione.class
ferrovie/FerroviaBinaryPersister.class
persone/
persone/BadFileFormatException.class
persone/Persona.class
persone/LeggiPersone.class
persone/LeggiPersoneConScanner.class
persone/LeggiPersoneConSplit.class
voli/
voli/Flight.class
voli/FlightPos.class
voli/BadFileFormatException.class
voli/LeggiVolo.class
LineaMI-B0.txt
```

Conseguenza:
Eclipse include automaticamente
la libreria necessaria

Ora il file da leggere è
interno al JAR, letto da
getResourceAsStream